SCIENCES SORBONNE UNIVERSITÉ

Examen réparti 1 DataCloud
Master 2 SAR

Jonathan Lejeune

Novembre 2023

2 heures – **Tout document papier interdit**
Tout appareil électronique interdit

- Le barème est sur 21 points et est donné à titre indicatif. La note finale sera bornée à 20 points.
- Lors de la correction il sera tenu compte de la simplicité et de la lisibilité de vos réponses
- S'abstenir de répondre est préférable à répondre une absurdité
- Vous trouverez en annexe de la documentation sur l'API de Spark (RDD + DStream)

Exercice 1 – Questions de cours (4 points)

Question 1 ½ point
Quel est le rôle de Hadoop YARN ?

Question 2 ½ point
Justifier pourquoi le HDFS privilège le débit à la latence.

Question 3 ½ point
En quoi consiste la fédération de HDFS ?

Question 4 ½ point
En scala, quelle est la différence sémantique entre `class C[T]` et `class C[+T]` ?

Question 5 ½ point
Soit le code scala suivant :

```
1 object Truc {
2         var x = 42
3         def f(a:Int, b:Int)=a*x+b
4 }
```

La fonction `f` est-elle pure ? **Justifiez**

Question 6 ½ point
Donner un cas d'usage du mot clé `implicit` en scala.

Question 7 ½ point
Qu'est ce que le driver dans une application Spark ?

Question 8 ½ point
Pourquoi dit-on qu'une application Spark s'exécute de manière paresseuse ?

Exercice 2 – PageRank (8,5 points)

Le PageRank est un algorithme conçu par Google. Il permet d'estimer la popularité des pages Web afin de pouvoir les classer dans le résultat d'une requête sur leur moteur de recherche. La popularité d'une page Web est un score et se mesure à partir d'une somme pondérée de lien qui pointent vers elle (via un lien cliquable par exemple). La pondération implique qu'une page a un PageRank d'autant plus important qu'est grande la somme des PageRanks des pages qui pointent vers elle. En prenant en entrée une liste de liens, i.e. un couple d'URL dont le premier élément est la source et le deuxième la cible, l'algorithme renvoie pour chaque URL son score de PageRank. Le code ci-dessous est une version simplifiée de cet algorithme qui se base sur un nombre d'itérations fixe. La fonction PageRank prend 4 arguments :

- un chemin vers des données textuelles où chaque ligne représente un lien entre deux URLs
- un chemin de sortie pour stocker le résultat
- un nombre d'itérations
- un SparkContext

Le but de cet exercice n'est pas de comprendre comment l'algorithme fonctionne mais d'en analyser son code vis à vis de Spark.

```
1  def pageRank(in:String, out:String, iters: Int, spark: SparkContext)={
2   val lines = spark.textFile(in)
3   val links = lines.map(_.split(" "))
4                    .map(a=> (a(0),a(1)))
5                    .distinct()
6                    .groupByKey()
7   links.cache()
8   var ranks = links.mapValues(v => 1.0)
9   for (i <- 1 to iters) {
10     val join = links.join(ranks)
11     val values = join.map(_._2)
12     val tmp = values.flatMap(
13            urls_rank => urls_rank._1.map(
14                            url => (url, urls_rank._2 / urls_rank._1.size)
15                                        )
16                              )
17     ranks = tmp.reduceByKey(_ + _)
18  }
19  ranks.saveAsTextFile(out)
20 }
```

Question 1 1½ points

Donnez les types des variables `lines`, `links`, `ranks`, `join`, `values` et `tmp`.

Question 2 ½ point

Combien de job spark seront exécutés par cette fonction ? **Justifiez**.

Question 3 1½ points

Pour rappel, la méthode `textfile` utilise exactement le même algorithme de découpage que Hadoop MapReduce (i.e. la classe `TextInputFormat`) vu en TP. Donnez le nombre de partitions de `lines` pour les configurations suivantes de `in` étant un chemin HDFS :

1. un fichier texte de 275 Mo
2. un répertoire contenant un fichier 155 Mo, un fichier de 80Mo et un fichier de 40 Mo
3. un répertoire contenant un fichier de 140 Mo et un fichier de 135 Mo

Justifiez vos réponses

Question 4 3 points

On suppose que l'appel à la méthode `textfile` de la ligne 2 engendre un RDD à 3 partitions et que le nombre d'itérations est égal à 2.

1. Dessiner le graphe de dépendance de RDD
2. y faire apparaître les stages
3. en déduire le graphe de tâche engendré par l'exécution de cette fonction.

Question 5 1 point

En exécutant cette fonction, on se rend compte via l'interface Web de Spark que le driver de cette application a fait une optimisation à l'instruction `join` de la ligne 10 en n'engendrant pas de nouveau stage. L'instruction join a donc été considérée ici comme une transformation simple bien qu'elle soit de base une transformation shuffle. Justifiez le fait que cette optimisation ai pu avoir lieu et qu'elle ne change pas la spécification de l'instruction `join`.

Question 6 1 point

La méthode `cache()` est équivalent à `persist(StorageLevel.MEMORY_ONLY)`. Pourquoi est-ce judicieux de l'appliquer sur la variable `links` ?

Exercice 3 – Utilisation d'un service en ligne (8,5 points)

Nous considérons un service en ligne où des utilisateurs doivent se connecter pour l'utiliser. L'utilisation du service commence au moment de la connexion d'un utilisateur et se termine lorsque cet utilisateur se déconnecte. Pour des raisons de répartition de charge le service est géo distribué sur N serveurs. Le serveur numéro i ($0<i \leq N$) a le nom *serveur_i*. Chaque serveur maintient chaque jour un fichier de log dans son système de fichier local afin de tracer les différentes connexions/déconnexions des utilisateurs qu'il a dû traiter. Lors du changement de journée, chaque serveur copie son fichier de log du jour terminé dans un dossier HDFS. La JVM maître du HDFS est déployée sur la machine *hdfs.codel.fr* et écoute sur le port 9000. Les fichiers de log sont stockés selon l'arborescence suivante :

- le dossier racine de l'arborescence se nomme *logs* et se trouve à la racine du HDFS.
- chaque sous-dossier du répertoire *logs* regroupe une année donnée.
- chaque dossier d'une année comporte un sous-dossier pour chaque mois écoulé
- chaque dossier d'un mois comporte un sous-dossier pour chaque jour du mois
- et enfin chaque dossier d'un jour comporte N fichiers de logs où chaque fichier correspond à ce qu'un serveur a logué dans la journée correspondante. Le nom de chaque fichier correspondra au nom du serveur qui l'a produit.

Par exemple si on prend $N = 3$ le dossier HDFS */logs/2021/12/10* regroupera les fichiers de log de la journée du 10 décembre 2021 et contiendra 3 fichiers : serveur_1, serveur_2 et serveur_3

La ligne d'un fichier de log produite par un serveur est composée de 4 mots séparés par un seul espace :

- le premier mot indique le nom du serveur sur lequel la connexion a eu lieu

- le deuxième mot indique le nom de l'utilisateur qui s'est connecté
- le troisième mot indique une estampille temporelle en millisecondes indiquant la date (depuis le 1/1/1970) à laquelle la connexion a débuté
- le quatrième mot indique une estampille temporelle en millisecondes indiquant la date (depuis le 1/1/1970) à laquelle la connexion s'est terminé, c'est à dire la date à laquelle l'utilisateur a cessé d'utiliser le service (déconnexion)

On donne ici un exemple de ligne :

```
serveur_2.datacloud.fr john_the_ripper 1541854959323 1541855074217
```

Pour avoir le temps d'une connexion il suffit donc de faire la soustraction entre la valeur du quatrième mot et la valeur du troisième mot.

Chaque jour, une fois que tous les serveurs ont envoyé leur fichier de log sur le HDFS, on souhaite produire dans le dossier HDFS correspondant, un fichier nommé *stat_day* contenant pour chaque utilisateur le temps d'utilisation total du service dans la journée concernée. Ainsi on exécutera le programme spark suivant :

```
1  object UtilisationService {
2    val hdfs_url = "hdfs://hdfs.datacloud.fr:9000"
3
4    def main(args:Array[String]):Unit={
5
6      val sparkconf = new SparkConf().setAppName("Application")
7      val hadoopconf = new Configuration
8      hadoopconf.set("fs.defaultFS", hdfs_url)
9      val sc= new SparkContext(sparkconf)
10     val fs = FileSystem.get(hadoopconf)
11
12     val annee = args(0)
13     val mois = args(1)
14     val jour = args(2)
15     val dir="/logs/"+annee+"/"+mois+"/"+jour
16
17     val list:Seq[String] = listFile(fs, dir)//retourne la liste des noms de fichiers
           contenu dans dir
18
19     produceStatDayFile(hdfs_url+dir, list.map(hdfs_url+_), sc)
20
21   }
22
23   def produceStatDayFile(outdir:String, files:Seq[String], sc:SparkContext):Unit={
24     val rdd:RDD[String] = multiTextFile(files,sc)
25     //a completer
26   }
27
28
29   def multiTextFile(files:Seq[String],sc:SparkContext):RDD[String]={
30     //a completer
31   }
32 }
```

Ainsi par exemple à la fin du 8 novembre 2023, on appellera ce programme avec les arguments 2023, 11 et 8.

Question 1 1½ points

La méthode `textfile` de l'API Spark permet de produire un `RDD[String]` à partir d'un fichier texte. Ceci est inadapté à notre problème car il est nécessaire ici de prendre en compte plusieurs fichiers textes (i.e. les fichiers de chaque serveur). Complétez le code de la méthode `multiTextFile` qui permet de produire un seul `RDD[String]` qui contiendra l'union de toutes les lignes des fichiers passés en paramètre de la fonction.

Question 2 2 points

Compléter la fonction `produceStatDayFile(outdir:String, files:Seq[String], sc:SparkContext )` qui produit dans le répertoire `outdir` un fichier `stat_day` indiquant pour chaque utilisateur le temps passé à utiliser le service. On assurera qu'un seul fichier soit produit, c'est à dire que le rdd sur lequel on applique la fonction `saveAsTextFile` ne contienne qu'une seule partition. Chaque ligne du fichier de sortie devra être composé de deux mots séparés par un espace : le premier étant l'id de l'utilisateur et le deuxième étant le le temps d'utilisation associé.

Question 3 2 points

On souhaiterait maintenir les mêmes informations mais pour chaque mois. Pour cela on considère qu'un fichier `stat_month` est présent dans le dossier du mois correspondant et est modifié tous les jours, à chaque fois qu'un fichier `stat_day` est créé. Lorsque le premier jour du mois se termine, on supposera que ce fichier sera créé à partir d'une copie du fichier `stat_day`. Notre but est donc de coder la méthode `updateStatMonth(url_stat_month:String, url_stat_day:String, sc :SparkContext):RDD[String]` où `url_stat_month` indique l'URL HDFS du fichier `stat_month` à mettre à jour, `url_stat_day` indique l'URL HDFS du fichier `stat_day` venant d'être produit et `sc` le sparkcontext qu'on utilisera. La méthode retourne le nouveau contenu de `stat_month` à partir des données contenu dans `stat_day` sous la forme d'un `RDD[String]` à une seule partition (on ne gérera pas la mise à jour effective du fichier qui se sera supposée faite par le code appelant de cette méthode). On considéra que les fichiers `stat_day` et les fichiers `stat_month` sont formatés de la mème manière. Écrire la méthode `updateStatMonth`.

On souhaite maintenant connaître en temps réel le **temps d'utilisation cumulé de chaque utilisateur** grâce à Spark Streaming. Désormais chaque serveur enverra chaque ligne de log produite vers un système d'agrégation de logs (type Kafka) qui le retransmettra ensuite vers notre système Spark.

Question 4 ½ point

Pourquoi est-il préférable de passer par un système intermédiaire d'agrégation de flux entre les serveurs et Spark Streaming, plutôt qu'une communication directe entre SparkStreaming et chaque serveur via une socket TCP ?

Question 5 2½ points

Écrire un programme SparkStreaming permettant de connaitre en temps réel les 10 utilisateurs qui utilisent le plus le service **depuis le début de l'acquisition du flux**. On affichera ce classement toute les 30 secondes. On supposera que la source d'émission du système d'agrégation est la machine *agreg.datacloud.fr* sur le port 4242 et que l'on lira ce flux via une socket TCP avec cette machine.

API for SparkContext	Meaning
new **SparkContext**(config: SparkConf)	Create a SparkContext that loads settings from a Spark Config object describing the application configuration. Any settings in this config overrides the default configs as well as system properties.
new **SparkContext**(master: String, appName: String, conf: SparkConf)	Alternative constructor that allows setting common Spark properties directly, : Cluster URL to connect to (e.g. mesos://host:port, spark://host:port, local[4]), a name for your application, to display on the cluster web UI and a SparkConf object specifying other Spark parameters
textFile(path: String, minPartitions: Int = defaultMinPartitions): RDD[String]	Read a text file from HDFS, a local file system (available on all nodes), or any Hadoop-supported file system URI, and return it as an RDD of Strings. Path is the path to the text file on a supported file system, minPartitions is the suggested minimum number of partitions for the resulting RDD. It returns RDD of lines of the text file
sequenceFile[K, V](path: String, keyClass: Class[K], valueClass: Class[V]): RDD[(K, V)]	Get an RDD for a Hadoop SequenceFile with given key and value types. Path is the directory to the input data files, the path can be comma separated paths as a list of inputs, keyClass (resp. valueClass) is the `Class` of the key (resp. the value) associated with `SequenceFileInputFormat`. It returns RDD of tuples of key and corresponding value.
parallelize[T](seq: Seq[T], numSlices: Int): RDD[T]	Distribute a local Scala collection to form an RDD. NumSlices is number of partitions to divide the collection into. It returns the RDD representing distributed collection
newAPIHadoopFile[K, V, F <: InputFormat[K, V]]path: String, fClass: Class[F], kClass: Class[K], vClass: Class[V], conf: Configuration = hadoopConfiguration): RDD[(K, V)]	Get an RDD for a given Hadoop file with an arbitrary new API InputFormat and extra configuration options to pass to the input format. Path is the directory to the input data files, the path can be comma separated paths as a list of inputs, fclass is the storage format of the data to be read , kClass (resp. vClass) is the `Class` of the key (resp. the value) Hadoop configuration. It returns RDD of tuples of key and corresponding value
broadcast[T](value: T): Broadcast[T]	Broadcast a read-only variable to the cluster, returning a org.apache.spark.broadcast.Broadcast object for reading it in distributed functions. The variable will be sent to each cluster only once.
collectionAccumulator[T]: CollectionAccumulator[T]	Create and register a `CollectionAccumulator`, which starts with empty list and accumulates inputs by adding them into the list.
doubleAccumulator: DoubleAccumulator **longAccumulator**: LongAccumulator	Create and register a double or long accumulator, which starts with 0 and accumulates inputs by `add`.
register(acc: AccumulatorV2[_, _]): Unit	Register the given accumulator. Accumulators must be registered before use, or it will throw exception.
stop(): Unit	Shut down the SparkContext.

Simple transformation for RDD[T]	Meaning
map[U](f: (T) =>U): RDD[U]	Return a new distributed dataset formed by passing each element of the source through a function *f* .
mapValues[U](f: V => U): RDD[(K, U)]	Pass each value in the key-value pair RDD through a map function without changing the keys this also retains the original RDD's partitioning.
filter(f: (T) =>Boolean): RDD[T]	Return a new dataset formed by selecting those elements of the source on which *f* returns true.
flatMap[U](f: (T) =>TraversableOnce[U]): RDD[U]	Similar to map, but each input item can be mapped to 0 or more output items (so *f* should return a Seq rather than a single item).
flatMapValues[U](f: V => TraversableOnce[U]): RDD[(K, U)]	Pass each value in the key-value pair RDD through a flatMap function without changing the keys ; this also retains the original RDD's partitioning.

union(other: RDD[T]): RDD[T]	Return a new dataset that contains the union of the elements in the source dataset and the argument.
zip[U](other: RDD[U]): RDD[(T, U)]	Zips this RDD with another one, returning key-value pairs with the first element in each RDD, second element in each RDD, etc. *Assumes that the two RDDs have the same number of partitions and the same number of elements in each corresponding partition* (e.g. one was made through a map on the other).
zipWithIndex(): RDD[(T, Long)]	Zips this RDD with its element indices. The ordering is first based on the partition index and then the ordering of items within each partition. So the first item in the first partition gets index 0, and the last item in the last partition receives the largest index. This is similar to Scala's zipWithIndex but it uses Long instead of Int as the index type. This method needs to trigger a spark job when this RDD contains more than one partitions.
keyBy[K](f: (T) =>K): RDD[(K, T)]	Creates tuples of the elements in this RDD by applying f
glom(): RDD[Array[T]]	Return an RDD created by coalescing all elements within each partition into an array.

Large transformation for RDD[T]	Meaning
intersection(other: RDD[T]): RDD[T] **intersection**(other: RDD[T], numPartitions: Int): RDD[T] **intersection**(other: RDD[T], part: Partitioner): RDD[T]	Return the intersection of this RDD and another one. The output will not contain any duplicate elements, even if the input RDDs did.
distinct(): RDD[T] **distinct**(numPartitions: Int): RDD[T]	Return a new dataset that contains the distinct elements of the source dataset.
cartesian[U](other: RDD[U]): RDD[(T, U)]	When called on datasets of types T and U, returns a dataset of (T, U) pairs (all pairs of elements).
repartition(numPartitions: Int): RDD[T]	Reshuffle the data in the RDD randomly to create either more or fewer partitions and balance it across them. This always shuffles all data over the network.
subtract(other: RDD[T]): RDD[T] **subtract**(other: RDD[T], numPartitions: Int): RDD[T] **subtract**(other: RDD[T], p: Partitioner): RDD[T]	Return an RDD with the elements from *this* that are not in *other*.
sortBy[K](f: (T) =>K, ascending: Boolean = true , numPartitions: Int = this.partitions.length): RDD[T]	Return this RDD sorted by the given key function.

Large transformation for RDD[T] and T= (K,V)	Meaning
groupByKey(): RDD[(K, Iterable[V])] **groupByKey**(numPartitions:Int): RDD[(K, Iterable[V])] **groupByKey**(p:Partitioner): RDD[(K, Iterable[V])]	When called on a dataset of (K, V) pairs, returns a dataset of (K, Iterable<V>) pairs.
reduceByKey(func: (V, V) =>V): RDD[(K, V)] **reduceByKey**(numPartitions:Int, func: (V, V) =>V): RDD[(K, V)] **reduceByKey**(partitioner: Partitioner, func: (V, V) =>V): RDD[(K, V)]	When called on a dataset of (K, V) pairs, returns a dataset of (K, V) pairs where the values for each key are aggregated using the given reduce function *func*, which must be of type (V,V) => V.
sortByKey(ascending: Boolean = true, numPartitions: Int = self.partitions.length) : RDD[(K, V)]	When called on a dataset of (K, V) pairs where K implements Ordered, returns a dataset of (K, V) pairs sorted by keys in ascending or descending order, as specified in the boolean `ascending` argument.

join[W](other: RDD[(K, W)]): RDD[(K, (V, W))] **join**[W](other: RDD[(K, W)], numPartitions: Int): RDD[(K, (V, W))] **join**[W](other: RDD[(K, W)], p: Partitioner): RDD[(K, (V, W))]	Return an RDD containing all pairs of elements with matching keys in *this* and *other*. Each pair of elements will be returned as a (k, (v1, v2)) tuple, where (k, v1) is in *this* and (k, v2) is in *other*
leftOuterJoin[W](other: RDD[(K, W)]): RDD[(K, (V, Option[W]))] **leftOuterJoin**[W](other: RDD[(K, W)], numPartitions: Int): RDD[(K, (V, Option[W]))] **leftOuterJoin**[W](other: RDD[(K, W)], partitioner: Partitioner): RDD[(K, (V, Option[W]))]	Perform a left outer join of `this` and `other`. For each element (k, v) in `this`, the resulting RDD will either contain all pairs (k, (v, Some(w))) for w in `other`, or the pair (k, (v, None)) if no elements in `other` have key k.
rightOuterJoin[W](other: RDD[(K, W)]): RDD[(K, (Option[V], W))] **rightOuterJoin**[W](other: RDD[(K, W)], numPartitions: Int): RDD[(K, (Option[V], W))] **rightOuterJoin**[W](other: RDD[(K, W)], partitioner: Partitioner): RDD[(K, (Option[V], W))]	Perform a right outer join of `this` and `other`. For each element (k, w) in `other`, the resulting RDD will either contain all pairs (k, (Some(v), w)) for v in `this`, or the pair (k, (None, w)) if no elements in `this` have key k.
fullOuterJoin[W](other: RDD[(K, W)]): RDD[(K, (Option[V], Option[W]))] **fullOuterJoin**[W](other: RDD[(K, W)], numPartitions: Int): RDD[(K, (Option[V], Option[W]))] **fullOuterJoin**[W](other: RDD[(K, W)], partitioner: Partitioner): RDD[(K, (Option[V], Option[W]))]	Perform a full outer join of `this` and `other`. For each element (k, v) in `this`, the resulting RDD will either contain all pairs (k, (Some(v), Some(w))) for w in `other`, or the pair (k, (Some(v), None)) if no elements in `other` have key k. Similarly, for each element (k, w) in `other`, the resulting RDD will either contain all pairs (k, (Some(v), Some(w))) for v in `this`, or the pair (k, (None, Some(w))) if no elements in `this` have key k.
cogroup[W](o: RDD[(K, W)]): RDD[(K, (Iterable[V], Iterable[W]))] **cogroup**[W](o: RDD[(K, W)], numPart: Int): RDD[(K, (Iterable[V], Iterable[W]))] **cogroup**[W](o: RDD[(K, W)], p: Partitioner): RDD[(K, (Iterable[V], Iterable[W]))]	When called on datasets of type (K, V) and (K, W), returns a dataset of (K, (Iterable<V>, Iterable<W>)) tuples. This operation is also called `groupWith`.
subtractByKey[W](other: RDD[(K, W)]): RDD[(K, V)] **subtractByKey**[W](other: RDD[(K, W)], numPartitions: Int): RDD[(K, V)] **subtractByKey**[W](other: RDD[(K, W)], p: Partitioner): RDD[(K, V)]	Return an RDD with the pairs from *this* whose keys are not in *other*

Action for RDD[T]	Meaning
reduce(f: (T, T) =>T): T	Aggregate the elements of the dataset using a function *func* (which takes two arguments and returns one). The function should be commutative and associative so that it can be computed correctly in parallel.
collect(): Array[T]	Return all the elements of the dataset as an array at the driver program. This is usually useful after a filter or other operation that returns a sufficiently small subset of the data.
count():Long	Return the number of elements in the dataset.
first(): T	Return the first element of the dataset (similar to take(1)).
take(num: Int): Array[T]	Return an array with the first *n* elements of the dataset.
top(num: Int)(implicit ord: Ordering[T]): Array[T]	Returns the top k (largest) elements from this RDD as defined by the specified implicit Ordering[T] and maintains the ordering.
saveAsObjectFile(path: String): Unit	Save this RDD as a SequenceFile of serialized objects.
saveAsTextFile(path: String): Unit	Save this RDD as a text file, using string representations of elements.
foreach(f: (T) =>Unit): Unit	Run a function *func* on each element of the dataset. This is usually done for side effects such as updating an Accumulator or interacting with external storage systems.

API for StreamingContext	Meaning
new **StreamingContext**(sparkContext: SparkContext, batchDuration: Duration)	Create a StreamingContext using an existing SparkContext. *BatchDuration is* the time interval at which streaming data will be divided into batches. Duration can be initialized by *Milliseconds(nb), Seconds(nb)* or *Minutes(nb)*
checkpoint(directory: String)	Set the context to periodically checkpoint the DStream operations for driver fault-tolerance. The directory is HDFS-compatible directory where the checkpoint data will be reliably stored. Note that this must be a fault-tolerant file system like HDFS.
socketTextStream(hostname: String, port: Int, sl: StorageLevel = StorageLevel.MEMORY_AND_DISK_SER_2): ReceiverInputDStream[String]	Creates an input stream from TCP source hostname:port. Data is received using a TCP socket and the receive bytes is interpreted as UTF8 encoded `\n` delimited lines. The *hostname* parameter is the host to connect to for receiving data, *port* is the port to connect to for receiving data, *sl* is the Storage level to use for storing the received objects.
socketStream[T: ClassTag](hostname: String, port: Int, converter: (InputStream) => Iterator[T], sl: StorageLevel): ReceiverInputDStream[T]	Create an input stream from network source hostname:port, where data is received as serialized blocks (serialized using the Spark's serializer) that can be directly pushed into the block manager without deserializing them. This is the most efficient way to receive data. The *hostname* parameter is the host to connect to for receiving data, *port* is the port to connect to for receiving data, *sl* is the Storage level to use for storing the received objects. T is the type of the objects in the received blocks.
textFileStream(directory: String): DStream[String]	Create an input stream that monitors a Hadoop-compatible filesystem for new files and reads them as text files (using key as LongWritable, value as Text and input format as TextInputFormat). Files must be written to the monitored directory by "moving" them from another location within the same file system. File names starting with . are ignored.
start(): Unit	Start the execution of the streams. Throws IllegalStateException if the StreamingContext is already stopped.
awaitTermination()	Wait for the execution to stop. Any exceptions that occurs during the execution will be thrown in this thread.
awaitTerminationOrTimeout(timeout: Long): Boolean	Wait for the execution to stop. Any exceptions that occurs during the execution will be thrown in this thread. The *timeout* parameter is the time to wait in milliseconds. Returns true if it's stopped; or throw the reported error during the execution; or false if the waiting time elapsed before returning from the method.
stop(stopSparkContext: Boolean = conf.getBoolean("spark.streaming.stopSparkContextByDefault", true)): Unit	Stop the execution of the streams immediately (does not wait for all received data to be processed). By default, if *stopSparkContext* is not specified, the underlying SparkContext will also be stopped. This implicit behavior can be configured using the SparkConf configuration *spark.streaming.stopSparkContextByDefault*. If *stopSparkContext* is true, it stops the associated SparkContext. The underlying SparkContext will be stopped regardless of whether this StreamingContext has been started.

Transformation for DStream[T]	Meaning
filter(filterFunc: (T) => Boolean): DStream[T]	Return a new DStream containing only the elements that satisfy a predicate.
map[U](mapFunc: (T) => U): DStream[U]	Return a new DStream by applying a function to all elements of this DStream
flatMap[U](flatMapFunc: (T) => TraversableOnce[U]): DStream[U]	Return a new DStream by applying a function to all elements of this DStream, and then flattening the results
glom(): DStream[Array[T]]	Return a new DStream in which each RDD is generated by applying glom() to each RDD of this DStream. Applying glom() to an RDD coalesces all elements within each partition into an array.
union(that: Dstream[T]): DStream[T]	Return a new DStream by unifying data of another DStream with this DStream.
repartition(numPartitions: Int): DStream[T]	Return a new DStream with an increased or decreased level of parallelism. Each RDD in the returned DStream has exactly numPartitions partitions.
count(): DStream[Long]	Return a new DStream in which each RDD has a single element generated by counting each RDD of this DStream.
countByValue(numPartitions: Int = ssc.sc.defaultParallelism)(implicit ord: Ordering[T] = null): DStream[(T, Long)]	Return a new DStream in which each RDD contains the counts of each distinct value in each RDD of this DStream. Hash partitioning is used to generate the RDDs with `numPartitions` partitions (Spark's default number of partitions if `numPartitions` not specified).
countByValueAndWindow(windowDuration: Duration, slideDuration: Duration, numPartitions: Int = ssc.sc.defaultParallelism)(implicit ord: Ordering[T] = null): DStream[(T, Long)]	Return a new DStream in which each RDD contains the count of distinct elements in RDDs in a sliding window over this DStream. Hash partitioning is used to generate the RDDs with `numPartitions` partitions (Spark's default number of partitions if `numPartitions` not specified).
transform[U](transformFunc: (RDD[T], Time) => RDD[U])(implicit arg0: ClassTag[U]): DStream[U]	Return a new DStream in which each RDD is generated by applying a function on each RDD of 'this' DStream.
reduce(reduceFunc: (T, T) => T): DStream[T]	Return a new DStream in which each RDD has a single element generated by reducing each RDD of this DStream.
foreachRDD(foreachFunc: (RDD[T]) => Unit): Unit	Apply a function to each RDD in this DStream. This is an output operator, so 'this' DStream will be registered as an output stream and therefore materialized.
countByWindow(windowDuration: Duration, slideDuration: Duration): DStream[Long]	Return a new DStream in which each RDD has a single element generated by counting the number of elements in a sliding window over this DStream. Hash partitioning is used to generate the RDDs with Spark's default number of partitions
reduceByWindow(reduceFunc: (T, T) => T, windowDuration: Duration, slideDuration: Duration): DStream[T]	Return a new DStream in which each RDD has a single element generated by reducing all elements in a sliding window over this DStream.
window(windowDuration: Duration, slideDuration: Duration): DStream[T]	Return a new DStream in which each RDD contains all the elements in seen in a sliding window of time over this DStream.

Transformation for DStream[T], T= (K,V)	Meaning
combineByKey[C](createCombiner: (V) => C, mergeValue: (C, V) => C, mergeCombiner: (C, C) => C, partitioner: Partitioner, mapSideCombine: Boolean = true): DStream[(K, C)]	Combine elements of each key in DStream's RDDs using custom functions. This is similar to the combineByKey for RDDs. Please refer to combineByKey in org.apache.spark.rdd.PairRDDFunctions in the Spark core documentation for more information.
reduceByKey(reduceFunc: (V, V) => V): DStream[(K, V)]	Return a new DStream by applying `reduceByKey` to each RDD. The values for each key are merged using the associative and commutative reduce function. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.
mapValues[U](mapValuesFunc: (V) => U): DStream[(K, U)]	Return a new DStream by applying a map function to the value of each key-value pairs in 'this' DStream without changing the key
flatMapValues[U](flatMapValuesFunc: (V) => TraversableOnce[U]): DStream[(K, U)]	Return a new DStream by applying a flatmap function to the value of each key-value pairs in 'this' DStream without changing the key.
join[W](other: DStream[(K, W)]): DStream[(K, (V, W))]	Return a new DStream by applying 'join' between RDDs of `this` DStream and `other` DStream. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.
leftOuterJoin[W](other: DStream[(K, W)]): DStream[(K, (V, Option[W]))]	Return a new DStream by applying 'left outer join' between RDDs of `this` DStream and `other` DStream.
fullOuterJoin[W](other: DStream[(K, W)]): DStream[(K, (Option[V], Option[W]))]	Return a new DStream by applying 'full outer join' between RDDs of `this` DStream and `other` DStream. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.
rightOuterJoin[W](other: DStream[(K, W)]): DStream[(K, (Option[V], W))]	Return a new DStream by applying 'right outer join' between RDDs of `this` DStream and `other` DStream. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.
cogroup[W](other: DStream[(K, W)])(implicit arg0: ClassTag[W]): DStream[(K, (Iterable[V], Iterable[W]))]	Return a new DStream by applying 'cogroup' between RDDs of `this` DStream and `other` DStream. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.
groupByKey(): DStream[(K, Iterable[V])]	Return a new DStream by applying `groupByKey` to each RDD. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.
groupByKeyAndWindow(windowDuration: Duration): DStream[(K, Iterable[V])]	Return a new DStream by applying `groupByKey` over a sliding window. This is similar to `DStream.groupByKey()` but applies it over a sliding window. The new DStream generates RDDs with the same interval as this DStream. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.
reduceByKeyAndWindow(reduceFunc: (V, V) => V, windowDuration: Duration): DStream[(K, V)]	Return a new DStream by applying `reduceByKey` over a sliding window on `this` DStream. Similar to `DStream.reduceByKey()`, but applies it over a sliding window. The new DStream generates RDDs with the same interval as this DStream. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.

reduceByKeyAndWindow(reduceFunc: (V, V) => V, invReduceFunc: (V, V) => V, windowDuration: Duration, slideDuration: Duration = self.slideDuration, numPartitions: Int = ssc.sc.defaultParallelism, filterFunc: ((K, V)) => Boolean = null): DStream[(K, V)]	Return a new DStream by applying incremental `reduceByKey` over a sliding window. The reduced value of over a new window is calculated using the old window's reduced value : 1. reduce the new values that entered the window (e.g., adding new counts) 2. "inverse reduce" the old values that left the window (e.g., subtracting old counts) This is more efficient than reduceByKeyAndWindow without "inverse reduce" function. However, it is applicable to only "invertible reduce functions". Hash partitioning is used to generate the RDDs with Spark's default number of partitions.
updateStateByKey[S](updateFunc: (Seq[V], Option[S]) => Option[S]):DStream[(K,S)]	Return a new "state" DStream where the state for each key is updated by applying the given function on the previous state of the key and the new values of each key. In every batch the updateFunc will be called for each state even if there are no new values. Hash partitioning is used to generate the RDDs with Spark's default number of partitions. If `updateFunc` function returns None, then corresponding state key-value pair will be eliminated.

Output Operation for DStream[T]	Meaning
foreachRDD(foreachFunc: (RDD[T]) => Unit): Unit	Apply a function to each RDD in this DStream. This is an output operator, so 'this' DStream will be registered as an output stream and therefore materialized.
print(num: Int): Unit	Print the first num elements of each RDD generated in this DStream. This is an output operator, so this DStream will be registered as an output stream and there materialized.
print(): Unit	Print the first ten elements of each RDD generated in this DStream. This is an output operator, so this DStream will be registered as an output stream and there materialized.
saveAsObjectFiles(prefix: String, suffix: String =""): Unit	Save each RDD in this DStream as a Sequence file of serialized objects. The file name at each batch interval is generated based on `prefix` and `suffix`: "prefix-TIME_IN_MS.suffix".
saveAsTextFiles(prefix: String, suffix: String = ""): Unit	Save each RDD in this DStream as at text file, using string representation of elements.
saveAsNewAPIHadoopFiles[F <: OutputFormat[K, V]](prefix: String, suffix: String): Unit	Save each RDD in `this` DStream as a Hadoop file. The file name at each batch interval is generated based on `prefix` and `suffix`: "prefix-TIME_IN_MS.suffix".